



An-Najah National University

Faculty of Engineering & Information Technolog

Distributed Operating Systems Project

Part 2

Students:

Hiba Burhan Banimeneh (12216976)

Shorouq Majdi Derieh (12217236)

Supervisor:

Dr.Samer Arandi

Introduction

This part builds upon the Bazar.com project developed in part 1 by introducing three key enhancements: replication, caching, and cache consistency. The goal is to improve the system's scalability and performance under increased workload. The new architecture incorporates in-memory caching for faster query responses, server replication with load balancing for high availability, and cache invalidation mechanisms to ensure data consistency.

System Architecture and Design

In Part 2, the system was extended by adding replication, caching, load balancing, and consistency mechanisms. The front-end server acts as the main entry point and handles request routing, in-memory caching, cache invalidation, and round-robin load balancing. Two catalog replicas store and manage book information, while two order replicas handle purchase requests. When a purchase occurs, the selected order replica updates both catalog replicas to keep the book quantities consistent. The front-end cache stores recently requested book information to improve response time, and it invalidates cached entries when updates occur to avoid stale data. The whole system is initialized using Docker Compose, which runs the front-end, catalog replicas, and order replicas on separate ports.

> System Initialization – all ports run:

```
PS D:\Bazar> docker-compose up --build
server instead.

order1 | * Running on all addresses (0.0.0.0)
order1 | * Running on http://127.0.0.1:5001

order1 | * Running on http://172.18.0.4:5001

order2 | Press CTRL+C to quit
order1 | Press CTRL+C to quit
frontend | * Serving Flask app 'frontend'
frontend | * Debug mode: off
frontend | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
frontend | * Running on all addresses (0.0.0.0)
frontend | * Running on http://127.0.0.1:5002
frontend | * Running on http://172.18.0.6:5002
frontend | Press CTRL+C to quit

View in Docker Desktop View Config Enable Watch Detach
catalog1 | Press CTRL+C to quit

order1 | * Serving Flask app 'order'
order2 | * Serving Flask app 'order'
order1 | * Debug mode: off
order2 | * Debug mode: off

order1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production
server instead.

order2 | WARNING: This is a development server. Do not use it in a production deployment. Use a production
server instead.

order1 | * Running on all addresses (0.0.0.0)
order1 | * Running on http://127.0.0.1:5001
```

```

PS D:\Bazar> docker-compose up --build
#31 [frontend] resolving provenance for metadata file
#31 DONE 0.0s
[+] up 5/5
  ✓ Image bazar-catalog1 Built 5.5s
  ✓ Image bazar-catalog2 Built 5.5s
[+] up 10/10ar-order1 Built 5.5s
  ✓ Image bazar-catalog1 Built 5.5s
  ✓ Image bazar-catalog2 Built 5.5s
  ✓ Image bazar-order1 Built 5.5s
  ✓ Image bazar-order2 Built 5.5s
  ✓ Image bazar-frontend Built 5.5s
  ✓ Container catalog2 Recreated 0.6s
  ✓ Container catalog1 Recreated 0.5s
  ✓ Container order2 Recreated 0.6s
  ✓ Container order1 Recreated 0.6s
  ✓ Container bazar-frontend-1 Recreated 0.7s
Attaching to catalog1, catalog2, frontend, order1, order2
catalog1 | * Serving Flask app 'catalog'
catalog2 | * Serving Flask app 'catalog'

```

>Load balancing round robin:

```

catalog2 | DEBUG: books = [(1, 'How to get a good grade in DOS in 40 minutes a day', 'distributed systems', 99.9, 101), (2, 'RPCs for Noobs', 'distributed systems', 50.0, 198)]
frontend | [Frontend] search topic='distributed systems'

catalog2 | Book Title: RPCs for Noobs, Book ID: 2 DOS in 40 minutes a day, Book ID: 1
catalog2 | 172.18.0.6 - - [24/May/2026 18:21:42] "GET /search/distributed%20systems HTTP/1.1" 200 -
frontend | [LB] Catalog request -> http://catalog1:5000

catalog1 | DEBUG: books = [(1, 'How to get a good grade in DOS in 40 minutes a day', 'distributed systems', 99.9, 101), (2, 'RPCs for Noobs', 'distributed systems', 50.0, 198)]
frontend | [Frontend] search topic='distributed systems'

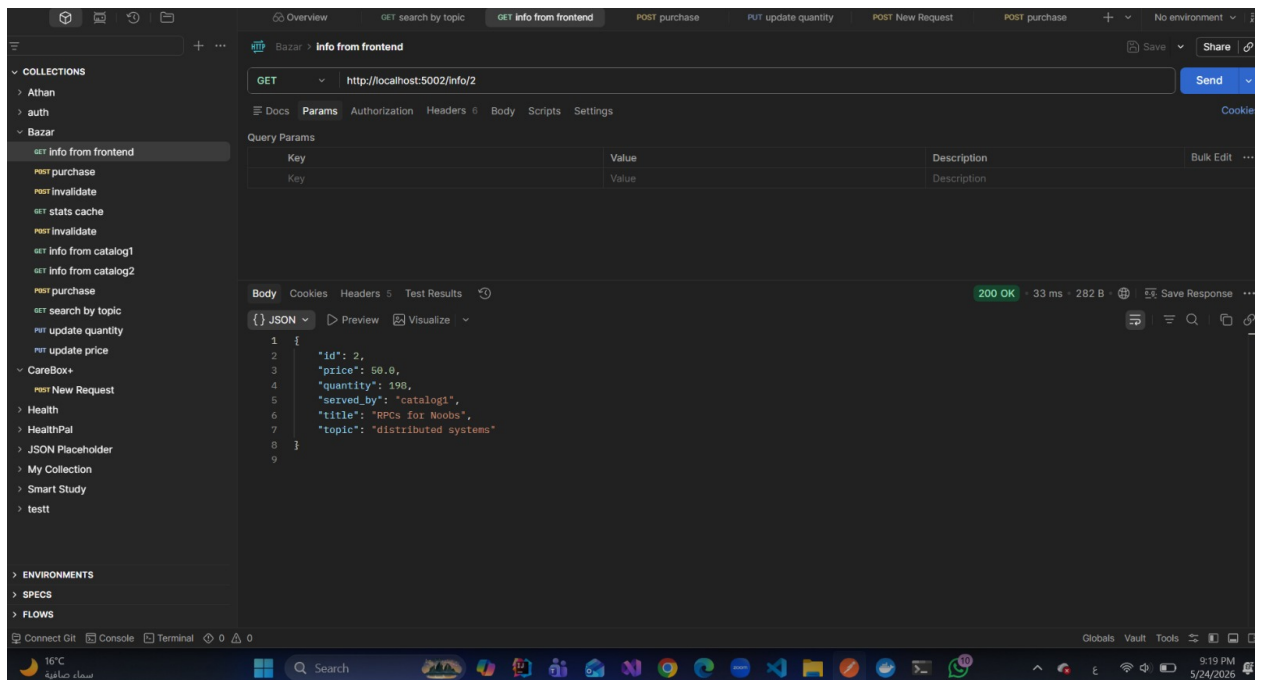
catalog1 | Book Title: RPCs for Noobs, Book ID: 2
catalog1 | 172.18.0.6 - - [24/May/2026 18:21:44] "GET /search/distributed%20systems HTTP/1.1" 200 -

```

Caching Mechanism

The front-end maintains an in-memory cache to serve repeated read requests quickly. Cached responses significantly reduce latency for queries that access the same book information multiple times.

->First GET request from server (33ms) – first get book 33ms from server:



The screenshot shows a REST client interface with a sidebar on the left containing a list of collections. The main panel displays a GET request to `http://localhost:5002/info/2`. The response is a 200 OK status with a response time of 33 ms and a body of 282 B. The response body is shown in JSON format:

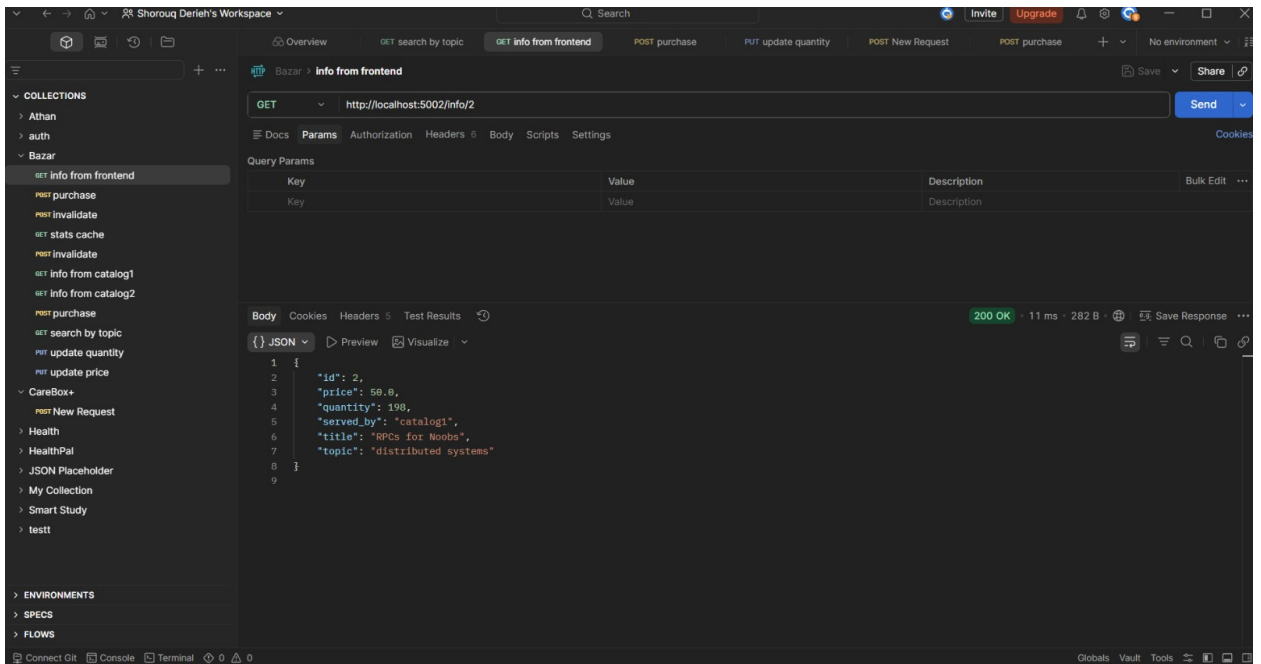
```
{
  "id": 2,
  "price": 50.0,
  "quantity": 198,
  "served_by": "catalog1",
  "title": "RPCs for Noobs",
  "topic": "distributed systems"
}
```

Below the REST client, a terminal window shows the output of the application. It includes a warning about using a development server in production, followed by log messages for the first GET request:

```
frontend | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSG
server instead.
frontend | * Running on all addresses (0.0.0.0)
frontend | * Running on http://127.0.0.1:5002
frontend | * Running on http://172.18.0.6:5002
frontend | Press CTRL+C to quit
frontend | [Cache] MISS book_id=2 | misses=1
frontend | [LB] Catalog request -> http://catalog1:5000

frontend | [Frontend] info id=2 -> http://catalog1:5000
frontend | [Cache] Stored book_id=2 | cache_size=1
frontend | 172.18.0.1 - - [24/May/2026 18:18:48] "GET /info/2 HTTP/1.1" 200 -
```

-> First GET request from cache (11ms) – first get book 11ms from cache:

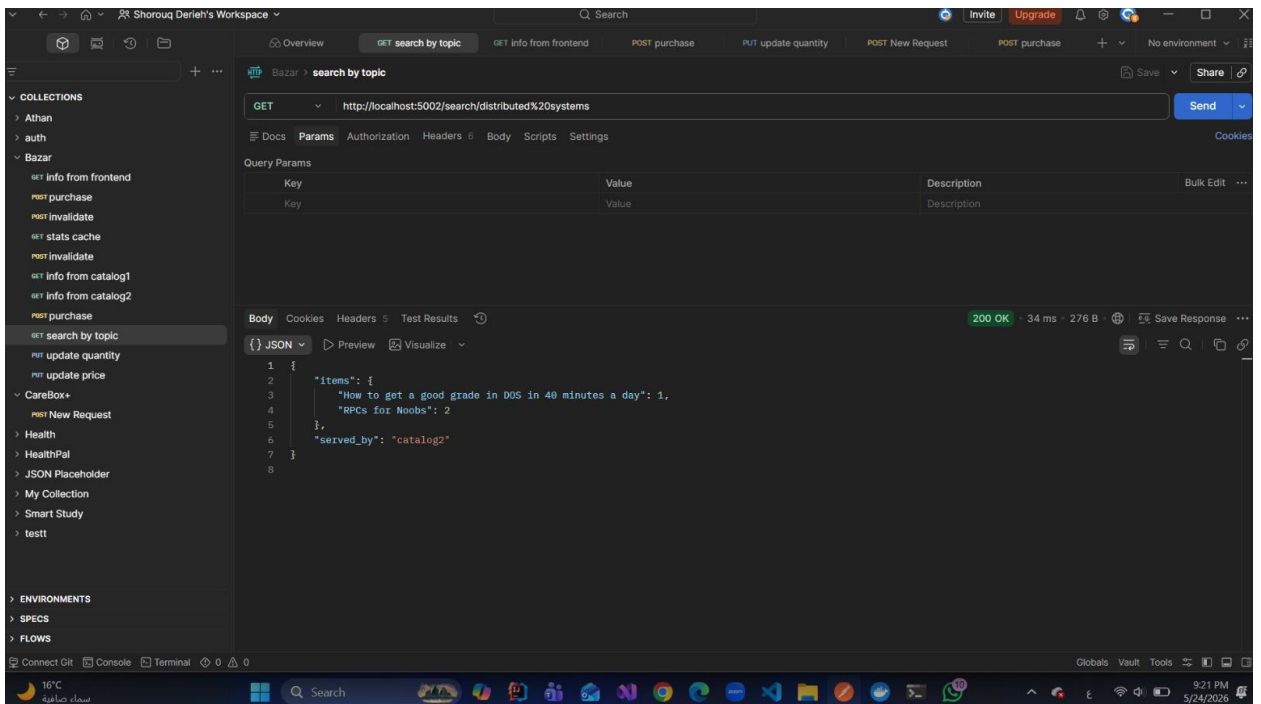


```

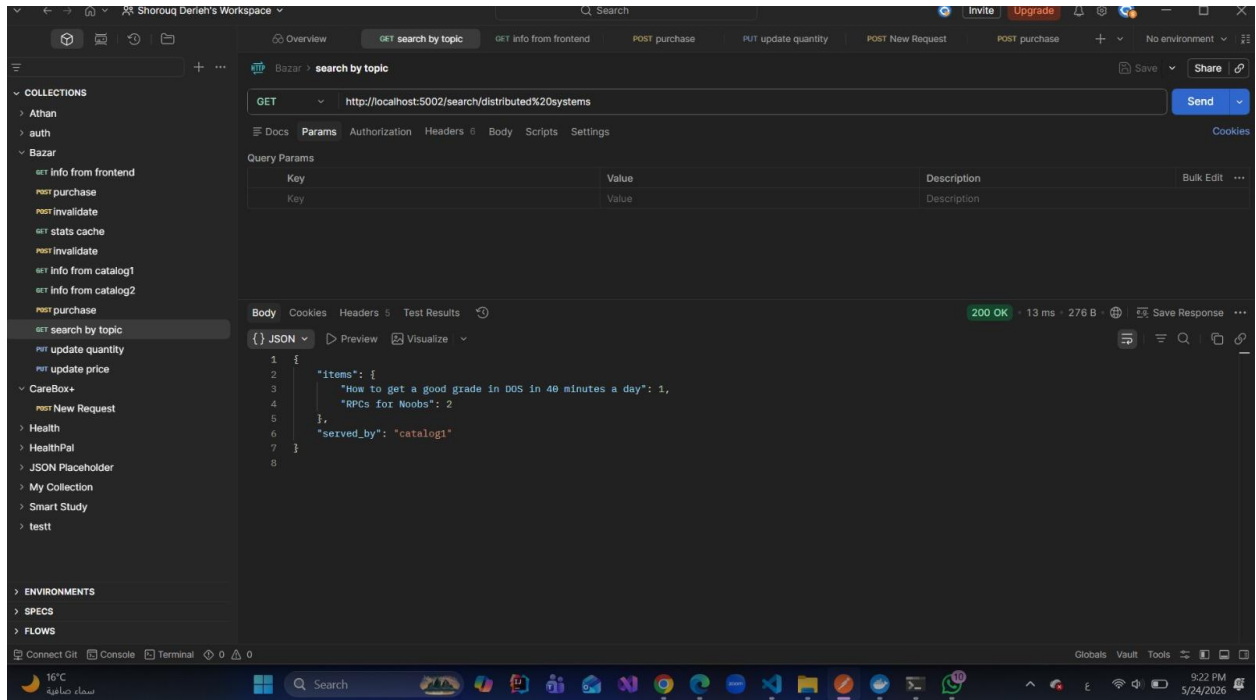
frontend | [Frontend] info id=2 -> http://catalog1:5000
frontend | [Cache] Stored book_id=2 | cache_size=1
frontend | 172.18.0.1 - - [24/May/2026 18:18:48] "GET /info/2 HTTP/1.1" 200 -
frontend | [Cache] HIT book_id=2 (age=86.43s) | hits=1
frontend | 172.18.0.1 - - [24/May/2026 18:20:15] "GET /info/2 HTTP/1.1" 200 -

```

-> Search request from server (34ms) – first search book 34ms from server:



-> search from cache (13ms) :

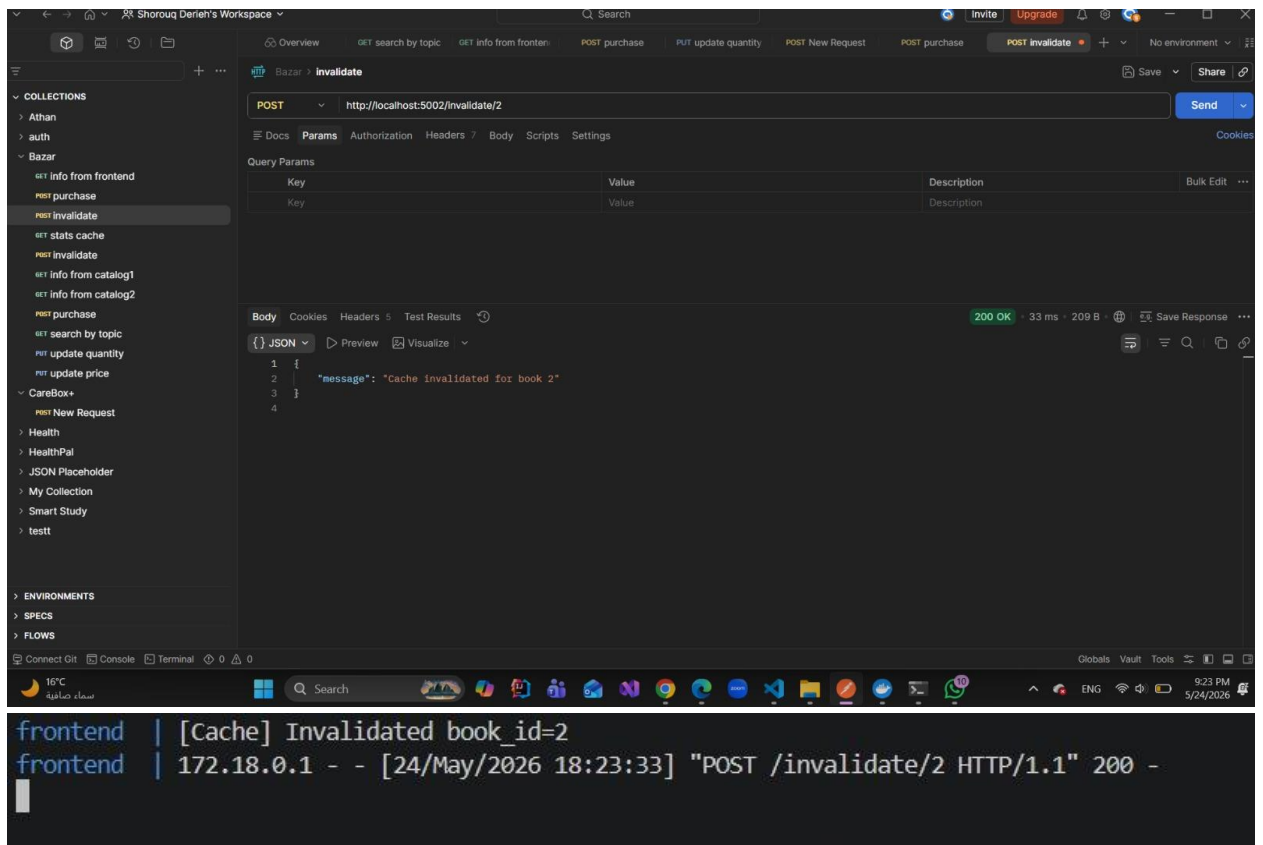


```
frontend | [Frontend] info id=2 -> http://catalog1:5000
frontend | [Cache] Stored book_id=2 | cache_size=1
frontend | 172.18.0.1 - - [24/May/2026 18:18:48] "GET /info/2 HTTP/1.1" 200 -
frontend | [Cache] HIT book_id=2 (age=86.43s) | hits=1
frontend | 172.18.0.1 - - [24/May/2026 18:20:15] "GET /info/2 HTTP/1.1" 200 -
```

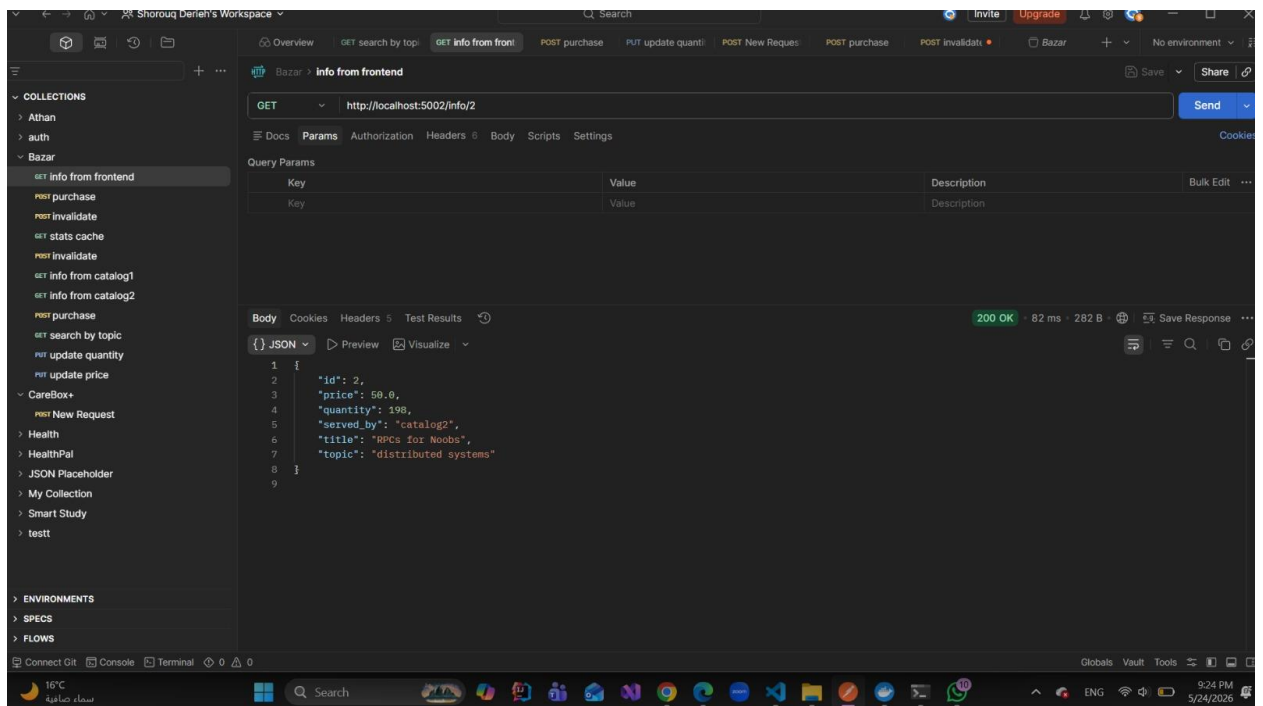
Cache Consistency

To maintain strong consistency, the system implements a cache invalidation protocol. When a write operation is performed (such as purchasing a book or updating inventory), the responsible replica sends an invalidate message to the cache before making the change.

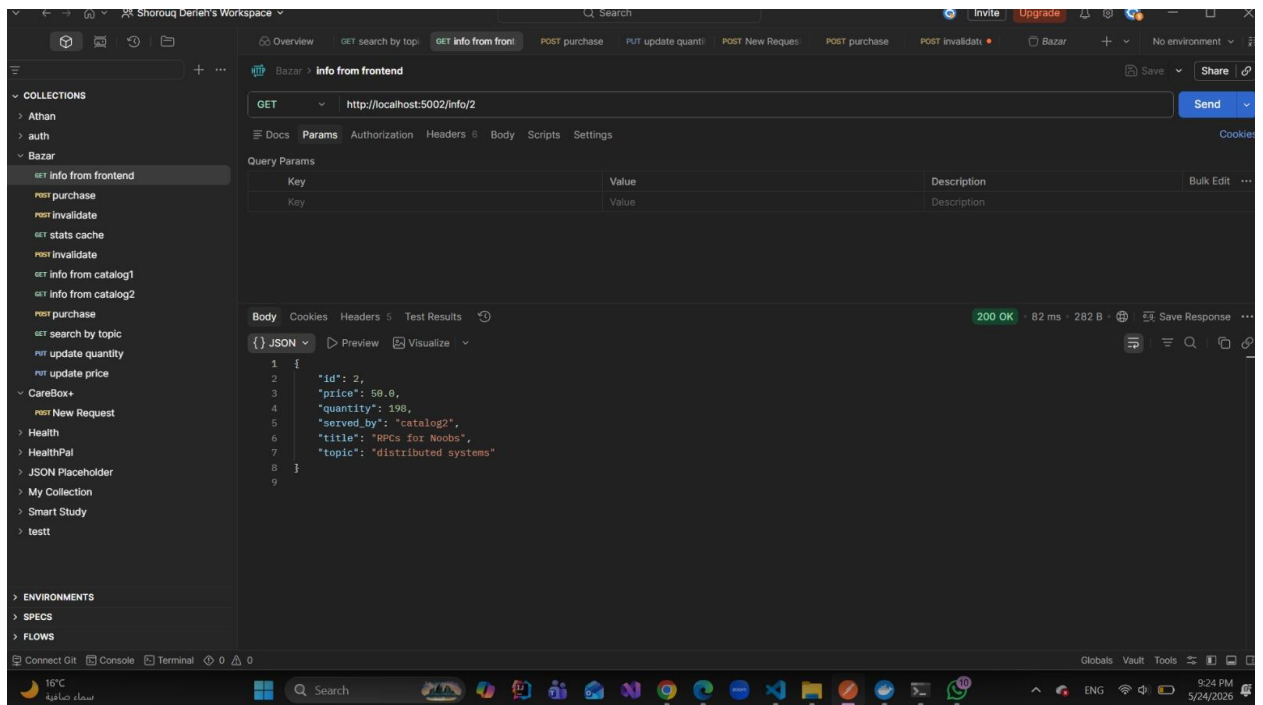
->Cache invalidated before update – invalidate cache:



->After cache invalidation, fresh data is fetched – after invalidta cache:



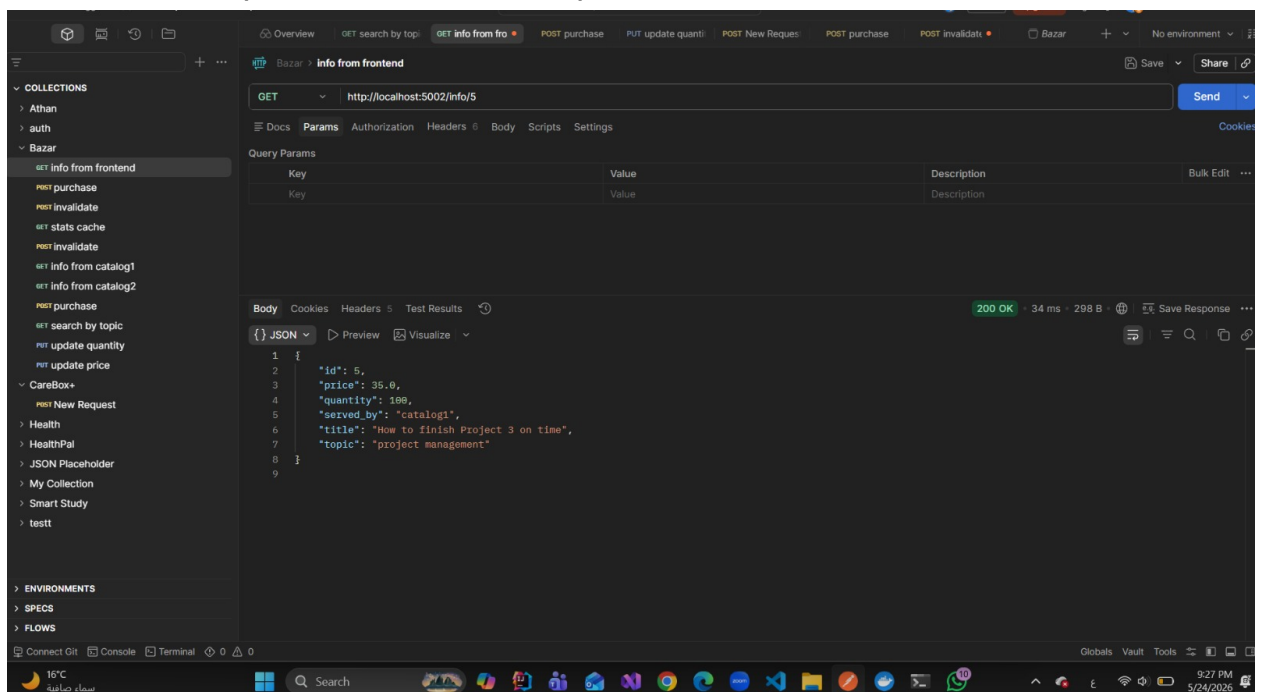
->Inconsistent data returned from stale cache – before make invalidate cache it return wrong stocks from cache:



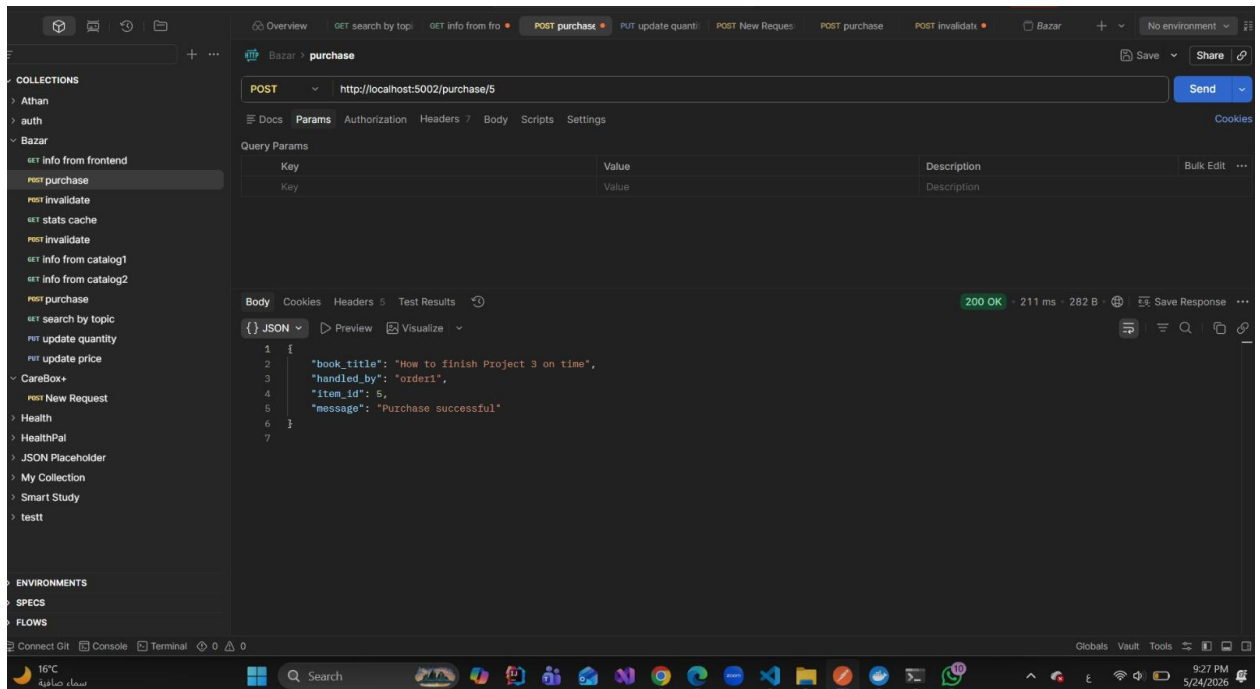
Order Processing and Inventory Updates

When a user purchases a book, the front-end routes the request to one of the order server replicas. The server checks availability and updates the stock accordingly. Cache is invalidated before stock updates to avoid serving outdated information.

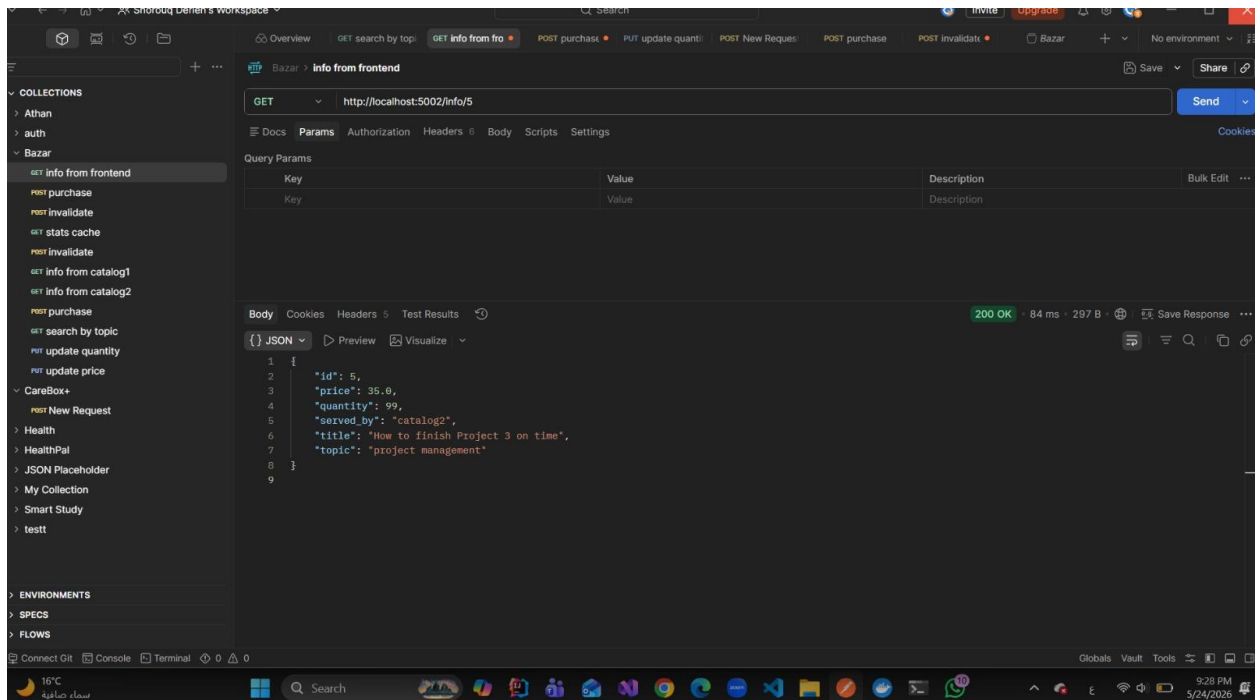
->Purchase request sent – order for purchase :



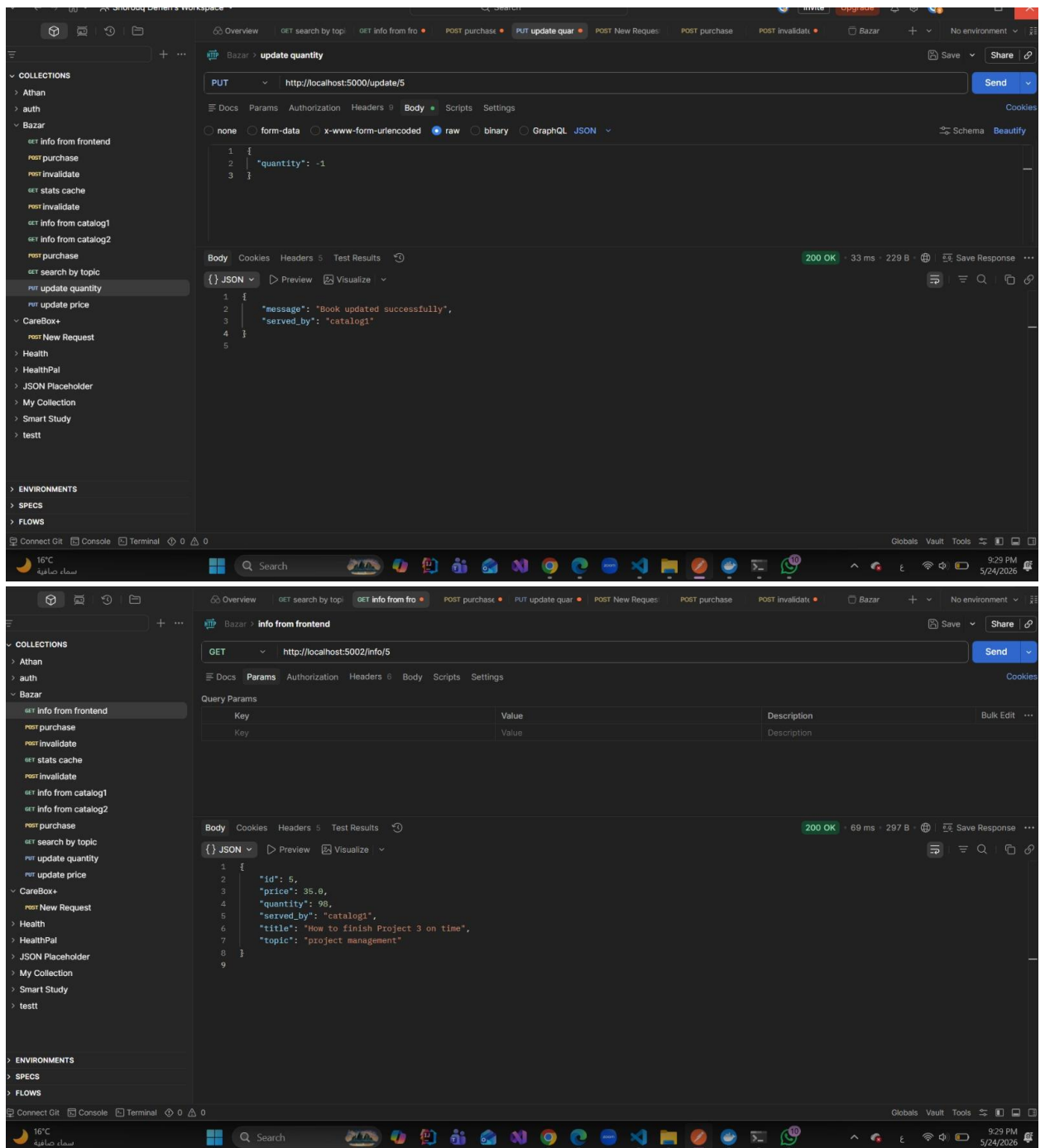
-> Successful book purchase – purchase book:



->The database update is performed using SQLite When a purchase is completed, the selected order server sends an update request to both catalog replicas to decrease the quantity of the purchased book by one



->Inventory updated for a book – update stock for book :



Experimental Results

```
Cache speedup: 1.7x

[Cache Stats from server]
{'cache_size': 1, 'hit_rate': '28.1%', 'hits': 25, 'misses': 64, 'total': 89}

Windows PowerShell
PS D:\Bazar> python measure.py

=====
EXPERIMENT 1: Cache MISS vs HIT response time
=====

[a] Cold requests (cache MISS) - 20 calls, clearing cache before each:
MISS avg=17.71 ms | median=14.41 ms | min=13.3 ms | max=49.08 ms

[b] Warm requests (cache HIT) - 20 calls after warm-up:
HIT : avg=10.61 ms | median=8.55 ms | min=7.49 ms | max=39.9 ms

--> Cache speedup: 1.7x faster with cache

=====
EXPERIMENT 2: Purchase response time (book id=5, qty=80)
=====
purchase: avg=61.65 ms | median=58.05 ms

=====
EXPERIMENT 3: Cache consistency - invalidation overhead
=====

[a] Cache populated for book_id=2
Invalidation avg=8.55 ms | median=8.4 ms

[b] First request after invalidation (should be MISS):
Post-invalidation MISS avg=14.27 ms

=====
PERFORMANCE SUMMARY TABLE
=====
Scenario                               Avg (ms)  Median (ms)
-----
Info - Cache MISS (cold)                17.71      14.41
Info - Cache HIT (warm)                 10.61       8.55
Purchase (write)                       61.65     58.05
Cache Invalidation                      8.55       8.4
Post-invalidation MISS                  14.27     14.24

Cache speedup: 1.7x
```

These results confirm that caching provides significant performance improvement, particularly for repetitive queries. Additionally, we observed the importance of cache invalidation. Without it, users might receive outdated stock information, leading to inconsistencies.

Discussion and Improvements

The system shows clear gains in response time using caching and can handle higher loads thanks to replication and load balancing. However, improvements could include: - Implementing Least Recently Used (LRU) cache eviction policy. - Adding write-through cache to reduce invalidation overhead. - Monitoring and dynamically adjusting load balancing based on replica health/performance.

Conclusion

This lab successfully demonstrates how caching, replication, and consistency mechanisms can greatly enhance the performance and reliability of distributed systems. Through careful cache management and load distribution, Bazar.com can now serve more users with better latency and consistent data.